Arquitectura de Software

Evolución de un Sistema de Facturación

Trabajo Práctico — 8vo Semestre de Ingeniería de Software

Contexto

El sistema actual es una aplicación monolítica simple que funciona por consola. El objetivo de este trabajo no es solo añadir funcionalidades, sino demostrar la capacidad de la Arquitectura Limpia para adaptarse a cambios tecnológicos drásticos sin comprometer las reglas de negocio.

Lo que se busca evaluar

1. Independencia de la entrada (desacoplamiento de UI)

El sistema debe ser capaz de procesar facturas provenientes de dos fuentes totalmente distintas: una API Web (síncrona) y un broker de mensajería (asíncrono, por ejemplo RabbitMQ). Se busca que el estudiante logre reutilizar la lógica de creación de facturas en ambos escenarios sin duplicar código y sin que el caso de uso "sepa" de dónde vienen los datos (HTTP o mensajería).

Aclaración sobre el punto de entrada

Se espera que ambos mecanismos de entrada (API REST y broker) sean **adaptadores simétricos sobre un mismo contrato de entrada de la capa de aplicación**, de modo que exista un único caso de uso para la creación de facturas y que dicho caso de uso no dependa del protocolo que lo invoca. El cliente que consume el sistema no conoce este contrato: interactúa con el protocolo (HTTP o AMQP), y es el adaptador correspondiente quien lo traduce e invoca la capa de aplicación.

2. Protección del dominio (reglas críticas de negocio)

Deberá implementarse una lógica simulada de:

- Control de fraude
- Políticas fiscales

Estas reglas deben residir exclusivamente en el núcleo del sistema (capa de dominio). Ni la base de datos ni la interfaz web podrán tener conocimiento de ellas. El sistema debe preservar su integridad independientemente de quién lo invoque.

3. Persistencia de datos genérica

El sistema deberá estar preparado para funcionar, sin cambios en su lógica de negocio, con PostgreSQL u Oracle como motor de base de datos. El cambio entre una tecnología y otra debe resolverse mediante configuración en la capa de infraestructura, demostrando que el negocio no depende de un proveedor de almacenamiento específico.

Aclaración sobre la persistencia y su simulación

El dominio debe definir el contrato de persistencia que necesita; las implementaciones concretas para PostgreSQL y Oracle residirán en la capa de infraestructura y serán intercambiables por configuración.
Las conexiones reales a las bases de datos pueden ser simuladas. Se acepta el uso de implementaciones en memoria (por ejemplo, repositorios con estructuras de datos volátiles), bases de datos embebidas (H2, SQLite, HSQLDB) o dobles de prueba, siempre que se demuestre que:

a) el dominio no conoce ni depende de la tecnología de persistencia concreta;

b) el intercambio entre proveedores se resuelve exclusivamente por configuración en la capa de infraestructura;

c) la lógica de negocio se ejecuta y se prueba sin necesidad de un motor de base de datos real.

Lo evaluado no es la conexión física a PostgreSQL u Oracle, sino la **capacidad de sustituir el adaptador de persistencia sin modificar el dominio ni la capa de aplicación**.

Objetivo final

Al terminar, el estudiante debe ser capaz de demostrar que puede cambiar el broker, la base de datos o la interfaz web sin modificar una sola línea de código en las capas de dominio y aplicación.

Entregables

- Código fuente.
- Diagrama de la Arquitectura Limpia.
- Informe ejecutivo que incluya lo realizado y las ventajas reales de la arquitectura.

Condiciones de entrega

El trabajo puede realizarse en grupos de hasta dos personas. En la defensa oral se valorará especialmente:

- El conocimiento profundo de la Arquitectura Limpia.
- La claridad en la exposición de los cambios implementados y su justificación.
- La correcta identificación y justificación del punto de entrada a la capa de aplicación que permite que ambos adaptadores (REST y broker) invoquen la misma lógica sin acoplarse a ella.
- La correcta identificación y justificación del contrato de persistencia que permite sustituir PostgreSQL por Oracle (o por una implementación simulada) sin modificar el dominio.